udemy - Troubleshooting and Optimization

Saturday, December 6, 2025 12:34 PM

Question 1

A data analytics company processes Internet-of-Things (IoT) data using Amazon Kinesis. The development team has noticed that the IoT data feed into Kinesis experiences periodic spikes. The PutRecords API call occasionally fails and the logs show that the failed call returns the response shown below:

```
HTTP/1.1 200 OK
x-amzn-RequestId: <RequestId>
Content-Type: application/x-amz-json-1.1
Content-Length: <PayloadSizeBytes>
Date: <Date>
{
   "FailedRecordCount": 2,
   "Records": [
      {
         "SequenceNumber": "4954346307654800757711050927030395603599752285183950126 86",
         "ShardId": "shardId-000000000000"
      },
      {
         "ErrorCode": "ProvisionedThroughputExceededException",
         "ErrorMessage": "Rate exceeded for shard shardId-000000000001 in stream exampleStreamName under account 111111111111."
      },
      {
         "ErrorCode": "InternalFailure",
         "ErrorMessage": "Internal service failure."
      }
   ]
}
```

As an AWS Certified Developer Associate, which of the following options would you recommend to address this use case? (Select two)

- Decrease the frequency or size of your requests
- Merge the shards to decrease the number of shards in the stream
- Decrease the number of KCL consumers
- Use an error retry and exponential backoff mechanism
- Increase the frequency or size of your requests

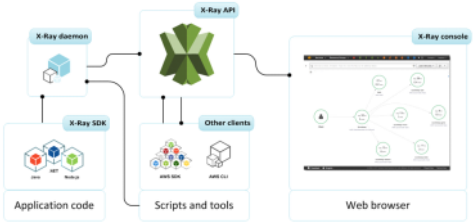
Overall explanation
Correct options:
Use an error retry and exponential backoff mechanism
Decrease the frequency or size of your requests
You can use PutRecords API call to write multiple data records into a Kinesis data stream in a single call. Each PutRecords request can support up to 500 records. Each record in the request can be as large as 1 MiB, up to a limit of 5 MiB for the entire request, including partition keys. Each shard can support writes up to 1,000 records per second, up to a maximum data write of 1 MiB per second.
The response Records array includes both successfully and unsuccessfully processed records. Kinesis Data Streams attempts to process all records in each PutRecords request. A single record failure does not stop the processing of subsequent records. As a result, PutRecords doesn't guarantee the ordering of records. An unsuccessfully processed record includes ErrorCode and ErrorMessage values. ErrorCode reflects the type of error and can be one of the following values: ProvisionedThroughputExceededException or InternalFailure. ProvisionedThroughputExceededException indicates that the request rate for the stream is too high, or the requested data is too large for the available throughput. Reduce the frequency or size of your requests.
To address the given use case, you can apply these best practices:
Reshard your stream to increase the number of shards in the stream.
Reduce the frequency or size of your requests.
Distribute read and write operations as evenly as possible across all of the shards in Data Streams.
Use an error retry and exponential backoff mechanism.
Incorrect options:
Merge the shards to decrease the number of shards in the stream
Increase the frequency or size of your requests
These two options contradict the explanation provided above, so these options are incorrect.
Decrease the number of KCL consumers - This option has been added as a distractor. The number of KCL consumers is irrelevant for the given use case since the ProvisionedThroughputExceededException is due to the PutRecords API call being used by the producers.

Question 2

You have deployed a Java application to an EC2 instance where it uses the X-Ray SDK. When testing from your personal computer, the application sends data to X-Ray but when the application runs from within EC2, the application fails to send data to X-Ray.
Which of the following does **NOT** help with debugging the issue?

- X-Ray sampling
- EC2 Instance Role
- CloudTrail
- EC2 X-Ray Daemon

Overall explanation
Correct option:
X-Ray sampling
By customizing sampling rules, you can control the amount of data that you record, and modify sampling behavior on the fly without modifying or redeploying your code. Sampling rules tell the X-Ray SDK how many requests to record for a set of criteria. X-Ray SDK applies a sampling algorithm to determine which requests get traced however because our application is failing to send data to X-Ray it does not help in determining the cause of failure.
X-Ray Overview:



via - https://docs.aws.amazon.com/xray/latest/devguide/aws-xray.html
Incorrect options:
EC2 X-Ray Daemon - The AWS X-Ray daemon is a software application that listens for traffic on UDP port 2000, gathers raw segment data, and relays it to the AWS X-Ray API. The daemon logs could help with figuring out the problem.
EC2 Instance Role - The X-Ray daemon uses the AWS SDK to upload trace data to X-Ray, and it needs AWS credentials with permission to do that. On Amazon EC2, the daemon uses the instance's instance profile role automatically. Eliminates API permission issues (in case the role doesn't have IAM permissions to write data to the X-Ray service)
CloudTrail - With CloudTrail, you can log, continuously monitor, and retain account activity related to actions across your AWS infrastructure. You can use AWS CloudTrail to answer questions such as - "Who made an API call to modify this resource?". CloudTrail provides event history of your AWS account activity thereby enabling governance, compliance, operational auditing, and risk auditing of your AWS account. You can check CloudTrail to see if any API call is being denied on X-Ray.

Question 3

While troubleshooting, a developer realized that the Amazon EC2 instance is unable to connect to the Internet using the Internet Gateway.
Which conditions should be met for Internet connectivity to be established? (Select two)
Your selection is correct

- The network ACLs associated with the subnet must have rules to allow inbound and outbound traffic
- The instance's subnet is not associated with any route table
- The instance's subnet is associated with multiple route tables with conflicting configurations
- The subnet has been configured to be Public and has no access to the internet
- The route table in the instance's subnet should have a route to an Internet Gateway

Overall explanation
Correct options:
The network ACLs associated with the subnet must have rules to allow inbound and outbound traffic - The network access control lists (ACLs) that are associated with the subnet must have rules to allow inbound and outbound traffic on port 80 (for HTTP traffic) and port 443 (for HTTPs traffic). This is a necessary condition for Internet Gateway connectivity
The route table in the instance's subnet should have a route to an Internet Gateway - A route table contains a set of rules, called routes, that are used to determine where network traffic from your subnet or gateway is directed. The route table in the instance's subnet should have a route defined to the Internet Gateway.
Incorrect options:
The instance's subnet is not associated with any route table - This is an incorrect statement. A subnet is implicitly associated with the main route table if it is not explicitly associated with a particular route table. So, a subnet is always associated with some route table.
The instance's subnet is associated with multiple route tables with conflicting configurations - This is an incorrect statement. A subnet can only be associated with one route table at a time.
The subnet has been configured to be Public and has no access to internet - This is an incorrect statement. Public subnets have access to the internet via Internet Gateway.

Question 4

A junior developer has been asked to configure access to an Amazon EC2 instance hosting a web application. The developer has configured a new security group to permit incoming HTTP traffic from 0.0.0.0/0 and retained any default outbound rules. A custom Network Access Control List (NACL) connected with the instance's subnet is configured to permit incoming HTTP traffic from 0.0.0.0/0 and retained any default outbound rules.
Which of the following solutions would you suggest if the EC2 instance needs to accept and respond to requests from the internet?

- An outbound rule on the security group has to be configured, to allow the response to be sent to the client on the HTTP port
- Outbound rules need to be configured both on the security group and on the NACL for sending responses to the Internet Gateway
- An outbound rule must be added to the Network ACL (NACL) to allow the response to be sent to the client on the ephemeral port range
- The configuration is complete on the EC2 instance for accepting and responding to requests

Overall explanation
Correct option:
An outbound rule must be added to the Network ACL (NACL) to allow the response to be sent to the client on the ephemeral port range
Security groups are stateful, so allowing inbound traffic to the necessary ports enables the connection. Network ACLs are stateless, so you must allow both inbound and outbound traffic. By default, each custom Network ACL denies all inbound and outbound traffic until you add rules.
To enable the connection to a service running on an instance, the associated network ACL must allow both: 1. Inbound traffic on the port that the service is listening on 2. Outbound traffic to ephemeral ports
When a client connects to a server, a random port from the ephemeral port range (1024-65535) becomes the client's source port.
The designated ephemeral port becomes the destination port for return traffic from the service. Outbound traffic to the ephemeral port must be allowed in the network ACL.
Incorrect options:
The configuration is complete on the EC2 instance for accepting and responding to requests - As explained above, this is an incorrect statement.
An outbound rule on the security group has to be configured, to allow the response to be sent to the client on the HTTP port - Security groups are stateful. Therefore you don't need a rule that allows responses to inbound traffic.
*Outbound rules need to be configured both on the security group and on the NACL for sending responses to the Internet Gateway** - Security Groups are stateful. Hence, return traffic is automatically allowed, so there is no need to configure an outbound rule on the security group.

Question 5

An Accounting firm extensively uses Amazon EBS volumes for persistent storage of application data of Amazon EC2 instances. The volumes are encrypted to protect the critical data of the clients. As part of managing the security credentials, the project manager has come across a policy snippet that looks like the following:

```
{
   "Version": "2012-10-17",
   "Statement": [
      {
         "Sid": "Allow for use of this Key",
         "Effect": "Allow",
         "Principal": {
            "AWS": "arn:aws:iam::111122223333:role/UserRole"
         },
         "Action": [
            "kms:GenerateDataKeyWithoutPlaintext",
            "kms:Decrypt"
         ],
         "Resource": "*"
      },
      {
         "Sid": "Allow for EC2 Use",
         "Effect": "Allow",
         "Principal": {
            "AWS": "arn:aws:iam::111122223333:role/UserRole"
         },
         "Action": [
            "kms:CreateGrant",
            "kms:ListGrants",
            "kms:RevokeGrant"
         ],
         "Resource": "*",
         "Condition": {
            "StringEquals": {
            "kms:ViaService": "ec2.us-west-2.amazonaws.com"
         }
      }
   ]
}
```

Which of the following options are correct regarding the policy?

- The first statement provides a specified IAM principal the ability to generate a data key and decrypt that data key from the CMK when necessary
- The second statement in this policy provides the security group (mentioned in first statement of the policy), the ability to create, list, and revoke grants for Amazon EC2
- The second statement in the policy mentions that all the resources stated in the first statement can take the specified role which will provide the ability to create, list, and revoke grants for Amazon EC2
- The first statement provides the security group the ability to generate a data key and decrypt that data key from the CMK when necessary

Overall explanation
Correct option:
The first statement provides a specified IAM principal the ability to generate a data key and decrypt that data key from the CMK when necessary - To create and use an encrypted Amazon Elastic Block Store (EBS) volume, you need permissions to use Amazon EBS. The key policy associated with the CMK would need to include these. The above policy is an example of one such policy.
In this CMK policy, the first statement provides a specified IAM principal the ability to generate a data key and decrypt that data key from the CMK when necessary. These two APIs are necessary to encrypt the EBS volume while it's attached to an Amazon Elastic Compute Cloud (EC2) instance.
The second statement in this policy provides the specified IAM principal the ability to create, list, and revoke grants for Amazon EC2. Grants are used to delegate a subset of permissions to AWS services, or other principals, so that they can use your keys on your behalf. In this case, the condition policy explicitly ensures that only Amazon EC2 can use the grants. Amazon EC2 will use them

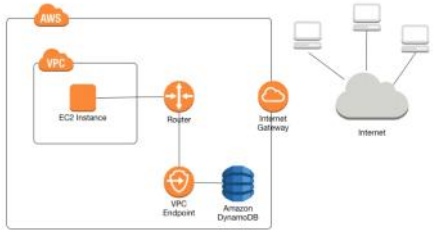
to re-attach an encrypted EBS volume back to an instance if the volume gets detached due to a planned or unplanned outage. These events will be recorded within AWS CloudTrail when, and if, they do occur for your auditing.
Incorrect options:
The first statement provides the security group the ability to generate a data key and decrypt that data key from the CMK when necessary
The second statement in this policy provides the security group (mentioned in the first statement of the policy), the ability to create, list, and revoke grants for Amazon EC2
The second statement in the policy mentions that all the resources stated in the first statement can take the specified role which will provide the ability to create, list, and revoke grants for Amazon EC2
These three options contradict the explanation provided above, so these options are incorrect.

Question 6

A CRM application is hosted on Amazon EC2 instances with the database tier using DynamoDB. The customers have raised privacy and security concerns regarding sending and receiving data across the public internet.
As a developer associate, which of the following would you suggest as an optimal solution for providing communication between EC2 instances and DynamoDB without using the public internet?

- Create an Internet Gateway to provide the necessary communication channel between EC2 instances and DynamoDB
- The firm can use a virtual private network (VPN) to route all DynamoDB network traffic through their own corporate network infrastructure
- Configure VPC endpoints for DynamoDB that will provide required internal access without using public internet
- Create a NAT Gateway to provide the necessary communication channel between EC2 instances and DynamoDB

Overall explanation
Correct option:
Configure VPC endpoints for DynamoDB that will provide required internal access without using public internet
When you create a VPC endpoint for DynamoDB, any requests to a DynamoDB endpoint within the Region (for example, dynamodb.us-west-2.amazonaws.com) are routed to a private DynamoDB endpoint within the Amazon network. You don't need to modify your applications running on EC2 instances in your VPC. The endpoint name remains the same, but the route to DynamoDB stays entirely within the Amazon network, and does not access the public internet. You use endpoint policies to control access to DynamoDB. Traffic between your VPC and the AWS service does not leave the Amazon network.
Using Amazon VPC Endpoints to Access DynamoDB:
The following diagram shows how an EC2 instance in a VPC can use a VPC endpoint to access DynamoDB.



via - https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/vpc-endpoints-dynamodb.html
Incorrect options:
The firm can use a virtual private network (VPN) to route all DynamoDB network traffic through their own corporate network infrastructure - You can address the requested security concerns by using a virtual private network (VPN) to route all DynamoDB network traffic through your own corporate network infrastructure. However, this approach can introduce bandwidth and availability challenges and hence is not an optimal solution here.
Create a NAT Gateway to provide the necessary communication channel between EC2 instances and DynamoDB - You can use a network address translation (NAT) gateway to enable instances in a private subnet to connect to the internet or other AWS services, but prevent the internet from initiating a connection with those instances. NAT Gateway is not useful here since the instance and DynamoDB are present in AWS network and do not need NAT Gateway for communicating with each other.
Create an Internet Gateway to provide the necessary communication channel between EC2 instances and DynamoDB - An internet gateway is a horizontally scaled, redundant, and highly available VPC component that allows communication between your VPC and the internet. An internet gateway serves two purposes: to provide a target in your VPC route tables for internet-routable traffic, and to perform network address translation (NAT) for instances that have been assigned public IPv4 addresses. Using an Internet Gateway would imply that the EC2 instances are connecting to DynamoDB using the public internet. Therefore, this option is incorrect.
References:
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/vpc-endpoints-dynamodb.html
https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html
https://docs.aws.amazon.com/vpc/latest/userguide/Carrier_Gateway.html

Question 7

You are a development team lead setting permissions for other IAM users with limited permissions. On the AWS Management Console, you created a dev group where new developers will be added, and on your workstation, you configured a developer profile. You would like to test that this user cannot terminate instances.
Which of the following options would you execute?

- Using the CLI, create a dummy EC2 and delete it using another CLI call
- Use the AWS CLI --test option
- Use the AWS CLI --dry-run option
- Retrieve the policy using the EC2 metadata service and use the IAM policy simulator

Overall explanation
Correct option:
Use the AWS CLI --dry-run option: The --dry-run option checks whether you have the required permissions for the action, without actually making the request, and provides an error response. If you have the required permissions, the error response is DryRunOperation, otherwise, it is UnauthorizedOperation.
Incorrect options:
Use the AWS CLI --test option - This is a made-up option and has been added as a distractor.
Retrieve the policy using the EC2 metadata service and use the IAM policy simulator - EC2 metadata service is used to retrieve dynamic information such as instance-id, local-hostname, public-hostname. This cannot be used to check whether you have the required permissions for the action.
Using the CLI, create a dummy EC2 and delete it using another CLI call - That would not work as the current EC2 may have permissions that the dummy instance does not have. If permissions were the same it can work but it's not as elegant as using the dry-run option.

Question 8

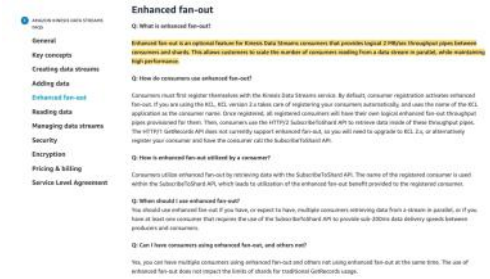
A data analytics company is processing real-time Internet-of-Things (IoT) data via Kinesis Producer Library (KPL) and sending the data to a Kinesis Data Streams driven application. The application has halted data processing because of a ProvisionedThroughputExceeded exception.
Which of the following actions would help in addressing this issue? (Select two)

- Increase the number of shards within your data streams to provide enough capacity
- Use Amazon SQS instead of Kinesis Data Streams
- Use Kinesis enhanced fan-out for Kinesis Data Streams
- Configure the data producer to retry with an exponential backoff
- Use Amazon Kinesis Agent instead of Kinesis Producer Library (KPL) for sending data to Kinesis Data Streams

Overall explanation
Correct option:
Configure the data producer to retry with an exponential backoff
Increase the number of shards within your data streams to provide enough capacity
Amazon Kinesis Data Streams enables you to build custom applications that process or analyze streaming data for specialized needs. You can continuously add various types of data such as clickstreams, application logs, and social media to an Amazon Kinesis data stream from hundreds of thousands of sources.
How Kinesis Data Streams Work

via - https://aws.amazon.com/kinesis/data-streams/

The capacity limits of an Amazon Kinesis data stream are defined by the number of shards within the data stream. The limits can be exceeded by either data throughput or the number of PUT records. While the capacity limits are exceeded, the put data call will be rejected with a ProvisionedThroughputExceeded exception.
If this is due to a temporary rise of the data stream's input data rate, retry (with exponential backoff) by the data producer will eventually lead to the completion of the requests.
If this is due to a sustained rise of the data stream's input data rate, you should increase the number of shards within your data stream to provide enough capacity for the put data calls to consistently succeed.
Incorrect options:
Use Amazon Kinesis Agent instead of Kinesis Producer Library (KPL) for sending data to Kinesis Data Streams - Kinesis Agent works with data producers. Using Kinesis Agent instead of KPL will not help as the constraint is the capacity limit of the Kinesis Data Stream.
Use Amazon SQS instead of Kinesis Data Streams - This is a distractor as using SQS will not help address the ProvisionedThroughputExceeded exception for the Kinesis Data Stream. This option does not address the issues in the use-case.
Use Kinesis enhanced fan-out for Kinesis Data Streams - You should use enhanced fan-out if you have, or expect to have, multiple consumers retrieving data from a stream in parallel. Therefore, using enhanced fan-out will not help address the ProvisionedThroughputExceeded exception as the constraint is the capacity limit of the Kinesis Data Stream.
Please review this note for more details on enhanced fan-out for Kinesis Data Streams:



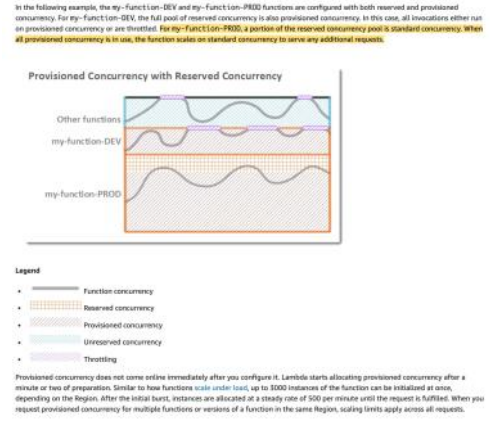
via - https://aws.amazon.com/kinesis/data-streams/faqs/

Question 9

The development team at a retail company is gearing up for the upcoming Thanksgiving sale and wants to make sure that the application's serverless backend running via Lambda functions does not hit latency bottlenecks as a result of the traffic spike.

- As a Developer Associate, which of the following solutions would you recommend to address this use-case?
- Configure Application Auto Scaling to manage Lambda provisioned concurrency on a schedule
- Add an Application Load Balancer in front of the Lambda functions
- No need to make any special provisions as Lambda is automatically scalable because of its serverless nature
- Configure Application Auto Scaling to manage Lambda reserved concurrency on a schedule

Overall explanation
Correct option:
Configure Application Auto Scaling to manage Lambda provisioned concurrency on a schedule
Concurrency is the number of requests that a Lambda function is serving at any given time. If a Lambda function is invoked again while a request is still being processed, another instance is allocated, which increases the function's concurrency.
Due to a spike in traffic, when Lambda functions scale, this causes the portion of requests that are served by new instances to have higher latency than the rest. To enable your function to scale without fluctuations in latency, use provisioned concurrency. By allocating provisioned concurrency before an increase in invocations, you can ensure that all requests are served by initialized instances with very low latency.
You can configure Application Auto Scaling to manage provisioned concurrency on a schedule or based on utilization. Use scheduled scaling to increase provisioned concurrency in anticipation of peak traffic. To increase provisioned concurrency automatically as needed, use the Application Auto Scaling API to register a target and create a scaling policy.
Please see this note for more details on provisioned concurrency:



via - https://docs.aws.amazon.com/lambda/latest/dg/configuration-concurrency.html
Incorrect options:
Configure Application Auto Scaling to manage Lambda reserved concurrency on a schedule - To ensure that a function can always reach a certain level of concurrency, you can configure the function with reserved concurrency. When a function has reserved concurrency, no other function can use that concurrency. More importantly, reserved concurrency also limits the maximum concurrency for the function, and applies to the function as a whole, including versions and aliases.
You cannot configure Application Auto Scaling to manage Lambda reserved concurrency on a schedule.
Add an Application Load Balancer in front of the Lambda functions - This is a distractor as just adding the Application Load Balancer will not help in scaling the Lambda functions to address the surge in traffic.
No need to make any special provisions as Lambda is automatically scalable because of its serverless nature - It's true that Lambda is serverless, however, due to the surge in traffic the Lambda functions can still hit the concurrency limits. So this option is incorrect.

Question 10

You create an Auto Scaling group to work with an Application Load Balancer. The scaling group is configured with a minimum size value of 5, a maximum value of 20, and the desired capacity value of 10. One of the 10 EC2 instances has been reported as unhealthy.
Which of the following actions will take place?

- The ASG will terminate the EC2 Instance
- The ASG will keep the instance running and re-start the application
- The ASG will format the root EBS drive on the EC2 instance and run the User Data again
- The ASG will detach the EC2 instance from the group, and leave it running

Overall explanation
Correct option:
The ASG will terminate the EC2 Instance
To maintain the same number of instances, Amazon EC2 Auto Scaling performs a periodic health check on running instances within an Auto Scaling group. When it finds that an instance is unhealthy, it terminates that instance and launches a new one. Amazon EC2 Auto Scaling creates a new scaling activity for terminating the unhealthy instance and then terminates it. Later, another scaling activity launches a new instance to replace the terminated instance.
Incorrect options:
The ASG will detach the EC2 instance from the group, and leave it running - The goal of the auto-scaling group is to get rid of the bad instance and replace it
The ASG will keep the instance running and re-start the application - The ASG does not have control of your application
The ASG will format the root EBS drive on the EC2 instance and run the User Data again - This will not happen, the ASG cannot assume the format of your EBS drive, and User Data only runs once at instance first boot.

Question 11

A Developer is configuring Amazon EC2 Auto Scaling group to scale dynamically.
Which metric below is NOT part of Target Tracking Scaling Policy?

ASGAverageCPUUtilization
ALBRequestCountPerTarget
ApproximateNumberOfMessagesVisible
ASGAverageNetworkOut

Overall explanation
Correct option:
ApproximateNumberOfMessagesVisible - This is a CloudWatch Amazon SQS queue metric. The number of messages in a queue might not change proportionally to the size of the Auto Scaling group that processes messages from the queue. Hence, this metric does not work for target tracking.
Incorrect options:
With target tracking scaling policies, you select a scaling metric and set a target value. Amazon EC2 Auto Scaling creates and manages the CloudWatch alarms that trigger the scaling policy and calculates the scaling adjustment based on the metric and the target value.
It is important to note that a target tracking scaling policy assumes that it should scale out your Auto Scaling group when the specified metric is above the target value. You cannot use a target tracking scaling policy to scale out your Auto Scaling group when the specified metric is below the target value.
ASGAverageCPUUtilization - This is a predefined metric for target tracking scaling policy. This represents the Average CPU utilization of the Auto Scaling group.
ASGAverageNetworkOut - This is a predefined metric for target tracking scaling policy. This represents the Average number of bytes sent out on all network interfaces by the Auto Scaling group.
ALBRequestCountPerTarget - This is a predefined metric for target tracking scaling policy. This represents the Number of requests completed per target in an Application Load Balancer target group.

Question 12

A serverless application built on AWS processes customer orders 24/7 using an AWS Lambda function and communicates with an external vendor's HTTP API for payment processing. The development team wants to notify the support team in near real-time using an existing Amazon Simple Notification Service (Amazon SNS) topic, but only when the external API error rate exceeds 5% of the total transactions processed in an hour.
As an AWS Certified Developer Associate, which option will you suggest as the most efficient solution?

- Configure and push high-resolution custom metrics to CloudWatch that record the failures of the external payment processing API calls. Create a CloudWatch alarm that sends a notification via the existing SNS topic when the error rate exceeds the specified rate
- Log the results of payment processing API calls to Amazon CloudWatch. Leverage Amazon CloudWatch Logs Insights to query the CloudWatch logs. Set up the Lambda function to check the output from CloudWatch Logs Insights on a schedule and send notification via the existing SNS topic when the error rate exceeds the specified rate
- Log the results of payment processing API calls to Amazon CloudWatch. Leverage Amazon CloudWatch Metric Filter to look at the CloudWatch logs. Set up the Lambda function to check the output from CloudWatch Metric Filter on a schedule and send notification via the existing SNS topic when the error rate exceeds the specified rate
- Configure CloudWatch metrics with detailed monitoring for the external payment processing API calls. Create a CloudWatch alarm that sends a notification via the existing SNS topic when the error rate exceeds the specified rate

Overall explanation
Correct option:
Configure and push high-resolution custom metrics to CloudWatch that record the failures of the external payment processing API calls. Create a CloudWatch alarm that sends a notification via the existing SNS topic when the error rate exceeds the specified rate
You can publish your own metrics, known as custom metrics, to CloudWatch using the AWS CLI or an API.
Each metric is one of the following:
Standard resolution, with data having a one-minute granularity
High resolution, with data at a granularity of one second
Metrics produced by AWS services are standard resolution by default. When you publish a custom metric, you can define it as either standard resolution or high resolution. When you publish a high-resolution metric, CloudWatch stores it with a resolution of 1 second, and you can read and retrieve it with a period of 1 second, 5 seconds, 10 seconds, 30 seconds, or any multiple of 60 seconds. High-resolution metrics can give you more immediate insight into your application's sub-minute activity. Keep in mind that every PutMetricData call for a custom metric is charged, so calling PutMetricData more often on a high-resolution metric can lead to higher charges.
You can create metric and composite alarms in Amazon CloudWatch. For the given use case, you can set up a CloudWatch metric alarm that watches the custom metric that captures the API errors and then triggers the alarm when the API error rate exceeds the 5% threshold. The alarm then sends a notification via the existing SNS topic.
Incorrect options:
Configure CloudWatch metrics with detailed monitoring for the external payment processing API calls. Create a CloudWatch alarm that sends a notification via the existing SNS topic when the error rate exceeds the specified rate - CloudWatch provides two categories of monitoring: basic monitoring and detailed monitoring. Detailed monitoring options differ based on the services that offer it. For example, Amazon EC2 detailed monitoring provides more frequent metrics, published at one-minute intervals, instead of the five-minute intervals used in Amazon EC2 basic monitoring. Detailed monitoring is offered by only some services. As explained above, you need to use custom metrics to capture data for the external payment processing API calls since detailed monitoring for the standard CloudWatch metrics cannot be used for this scenario.
Log the results of payment processing API calls to Amazon CloudWatch. Leverage Amazon CloudWatch Logs Insights to query the CloudWatch logs. Set up the Lambda function to check the output from CloudWatch Logs Insights on a schedule and send notification via the existing SNS topic when the error rate exceeds the specified rate - CloudWatch Logs Insights enables you to interactively search and analyze your log data in Amazon CloudWatch Logs. You can perform queries to help you more efficiently and effectively respond to operational issues. This option is not the right fit for the given use case since Lambda cannot monitor the output of the CloudWatch Logs Insights on a real-time basis since it is being invoked on a schedule. Also, it is not an efficient solution since Lambda will need significant custom code to parse and compute the external API error rate from the CloudWatch Logs Insights data.
Log the results of payment processing API calls to Amazon CloudWatch. Leverage Amazon CloudWatch Metric Filter to look at the CloudWatch logs. Set up the Lambda function to check the output from CloudWatch Metric Filter on a schedule and send notification via the existing SNS topic when the error rate exceeds the specified rate - You can search and filter the log data coming into CloudWatch Logs by creating one or more metric filters. Metric filters define the terms and patterns to look for in log data as it is sent to CloudWatch Logs. CloudWatch Logs uses these metric filters to turn log data into numerical CloudWatch metrics that you can graph or set an alarm on. This option is not the best fit for the given use case since Lambda cannot monitor the output of the CloudWatch Metric Filter on a real-time basis since it is being invoked on a schedule. Also, it is not an efficient solution since Lambda will need significant custom code to parse and compute the external API error rate from the CloudWatch Metric Filter data.

Question 13

A developer from your team has configured the load balancer to route traffic equally between instances or across Availability Zones. However, Elastic Load Balancing (ELB) routes more traffic to one instance or Availability Zone than the others.
Why is this happening and how can it be fixed? (Select two)
Your selection is incorrect

For Application Load Balancers, cross-zone load balancing is disabled by default

- After you disable an Availability Zone, the targets in that Availability Zone remain registered with the load balancer, thereby receiving random bursts of traffic
- There could be short-lived TCP connections between clients and instances
- Instances of a specific capacity type aren't equally distributed across Availability Zones
- Sticky sessions are enabled for the load balancer

Overall explanation
Correct option:
Sticky sessions are enabled for the load balancer - This can be the reason for potential unequal traffic routing by the load balancer. Sticky sessions are a mechanism to route requests to the same target in a target group. This is useful for servers that maintain state information in order to provide a continuous experience to clients. To use sticky sessions, the clients must support cookies. When a load balancer first receives a request from a client, it routes the request to a target, generates a cookie named AWSALB that encodes information about the selected target, encrypts the cookie, and includes the cookie in the response to the client. The client should include the cookie that it receives in subsequent requests to the load balancer. When the load balancer receives a request from a client that contains the cookie, if sticky sessions are enabled for the target group and the request goes to the same target group, the load balancer detects the cookie and routes the request to the same target.
If you use duration-based session stickiness, configure an appropriate cookie expiration time for your specific use case. If you set session stickiness from individual applications, use session cookies instead of persistent cookies where possible.
Instances of a specific capacity type aren't equally distributed across Availability Zones - A Classic Load Balancer with HTTP or HTTPS listeners might route more traffic to higher-capacity

instance types. This distribution aims to prevent lower-capacity instance types from having too many outstanding requests. It's a best practice to use similar instance types and configurations to reduce the likelihood of capacity gaps and traffic imbalances.
A traffic imbalance might also occur if you have instances of similar capacities running on different Amazon Machine Images (AMIs). In this scenario, the imbalance of the traffic in favor of higher-capacity instance types is desirable.
Incorrect options:
There could be short-lived TCP connections between clients and instances - This is an incorrect statement. Long-lived TCP connections between clients and instances can potentially lead to unequal distribution of traffic by the load balancer. Long-lived TCP connections between clients and instances cause uneven traffic load distribution by design. As a result, new instances take longer to reach connection equilibrium. Be sure to check your metrics for long-lived TCP connections that might be causing routing issues in the load balancer.
For Application Load Balancers, cross-zone load balancing is disabled by default - This is an incorrect statement. With Application Load Balancers, cross-zone load balancing is always enabled.
After you disable an Availability Zone, the targets in that Availability Zone remain registered with the load balancer, thereby receiving random bursts of traffic - This is an incorrect statement. After you disable an Availability Zone, the targets in that Availability Zone remain registered with the load balancer. However, even though they remain registered, the load balancer does not route traffic to them.

Question 14

As a Senior Developer, you manage 10 Amazon EC2 instances that make read-heavy database requests to the Amazon RDS for PostgreSQL. You need to make this architecture resilient for disaster recovery.
Which of the following features will help you prepare for database disaster recovery? (Select two)

- Enable the automated backup feature of Amazon RDS in a multi-AZ deployment that creates backups across multiple Regions
- Enable the automated backup feature of Amazon RDS in a multi-AZ deployment that creates backups in a single AWS Region
- Use RDS Provisioned IOPS (SSD) Storage in place of General Purpose (SSD) Storage
- Use database cloning feature of the RDS DB cluster
- Use cross-Region Read Replicas

Overall explanation
Correct option:
Use cross-Region Read Replicas
In addition to using Read Replicas to reduce the load on your source DB instance, you can also use Read Replicas to implement a DR solution for your production DB environment. If the source DB instance fails, you can promote your Read Replica to a standalone source server. Read Replicas can also be created in a different Region than the source database. Using a cross-Region Read Replica can help ensure that you get back up and running if you experience a regional availability issue.
Enable the automated backup feature of Amazon RDS in a multi-AZ deployment that creates backups in a single AWS Region
Amazon RDS provides high availability and failover support for DB instances using Multi-AZ deployments. Amazon RDS uses several different technologies to provide failover support. Multi-AZ deployments for MariaDB, MySQL, Oracle, and PostgreSQL DB instances use Amazon's failover technology.
The automated backup feature of Amazon RDS enables point-in-time recovery for your database instance. Amazon RDS will backup your database and transaction logs and store both for a user-specified retention period. If it's a Multi-AZ configuration, backups occur on the standby to reduce I/O impact on the primary. Automated backups are limited to a single AWS Region while manual snapshots and Read Replicas are supported across multiple Regions.
Incorrect options:
Enable the automated backup feature of Amazon RDS in a multi-AZ deployment that creates backups across multiple Regions - This is an incorrect statement. Automated backups are limited to a single AWS Region while manual snapshots and Read Replicas are supported across multiple Regions.
Use RDS Provisioned IOPS (SSD) Storage in place of General Purpose (SSD) Storage - Amazon RDS Provisioned IOPS Storage is an SSD-backed storage option designed to deliver fast, predictable, and consistent I/O performance. This storage type enhances the performance of the RDS database, but this isn't a disaster recovery option.
Use database cloning feature of the RDS DB cluster - This option has been added as a distractor. Database cloning is only available for Aurora and not for RDS.

Question 15

As a developer, you are looking at creating a custom configuration for Amazon EC2 instances running in an Auto Scaling group. The solution should allow the group to auto-scale based on the metric of 'average RAM usage' for your Amazon EC2 instances.
Which option provides the best solution?

- Enable detailed monitoring for EC2 and ASG to get the RAM usage data and create a CloudWatch Alarm on top of it
- Create a custom alarm for your ASG and make your instances trigger the alarm using PutAlarmData API
- Migrate your application to AWS Lambda
- Create a custom metric in CloudWatch and make your instances send data to it using PutMetricData. Then, create an alarm based on this metric

Overall explanation
Correct option:
Create a custom metric in CloudWatch and make your instances send data to it using PutMetricData. Then, create an alarm based on this metric - You can create a custom CloudWatch metric for your EC2 Linux instance statistics by creating a script through the AWS Command Line Interface (AWS CLI). Then, you can monitor that metric by pushing it to CloudWatch.
You can publish your own metrics to CloudWatch using the AWS CLI or an API. Metrics produced by AWS services are standard resolution by default. When you publish a custom metric, you can define it as either standard resolution or high resolution. When you publish a high-resolution metric, CloudWatch stores it with a resolution of 1 second, and you can read and retrieve it with a period of 1 second, 5 seconds, 10 seconds, 30 seconds, or any multiple of 60 seconds.
High-resolution metrics can give you more immediate insight into your application's sub-minute activity. But, every PutMetricData call for a custom metric is charged, so calling PutMetricData more often on a high-resolution metric can lead to higher charges.
Incorrect options:
Create a custom alarm for your ASG and make your instances trigger the alarm using PutAlarmData API - This solution will not work, your instances must be aware of each other's RAM utilization status, to know when the average RAM would be too high to trigger the alarm.
Enable detailed monitoring for EC2 and ASG to get the RAM usage data and create a CloudWatch Alarm on top of it - By enabling detailed monitoring you define the frequency at which the metric data has to be sent to CloudWatch, from 5 minutes to 1-minute frequency window. But, you still need to create and collect the custom metric you wish to track.
Migrate your application to AWS Lambda - This option has been added as a distractor. You cannot use Lambda for the given use-case.

Question 16

A junior developer working on ECS instances terminated a container instance in Amazon Elastic Container Service (Amazon ECS) as per instructions from the team lead. But the container instance continues to appear as a resource in the ECS cluster.
As a Developer Associate, which of the following solutions would you recommend to fix this behavior?

- You terminated the container instance while it was in STOPPED state, that lead to this synchronization issues
- The container instance has been terminated with AWS CLI, whereas, for ECS instances, Amazon ECS CLI should be used to avoid any synchronization issues
- A custom software on the container instance could have failed and resulted in the container hanging in an unhealthy state till restarted again
- You terminated the container instance while it was in RUNNING state, that lead to this synchronization issues

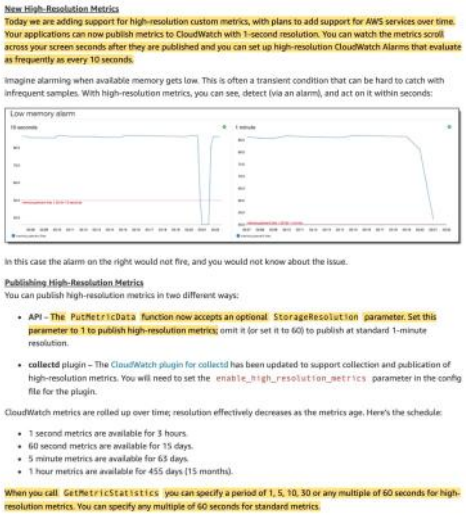
Overall explanation
Correct option:
You terminated the container instance while it was in STOPPED state, that lead to this synchronization issues - If you terminate a container instance while it is in the STOPPED state, that container instance isn't automatically removed from the cluster. You will need to deregister your container instance in the STOPPED state by using the Amazon ECS console or AWS Command Line Interface. Once deregistered, the container instance will no longer appear as a resource in your Amazon ECS cluster.
Incorrect options:
You terminated the container instance while it was in RUNNING state, that lead to this synchronization issues - This is an incorrect statement. If you terminate a container instance in the RUNNING state, that container instance is automatically removed, or deregistered, from the cluster.
The container instance has been terminated with AWS CLI, whereas, for ECS instances, Amazon ECS CLI should be used to avoid any synchronization issues - This is incorrect and has been added as a distractor.
A custom software on the container instance could have failed and resulted in the container hanging in an unhealthy state till restarted again - This is an incorrect statement. It is already mentioned in the question that the developer has terminated the instance.

Question 17

A telecommunications company that provides internet service for mobile device users maintains over 100 c4.large instances in the us-east-1 region. The EC2 instances run complex algorithms. The manager would like to track CPU utilization of the EC2 instances as frequently as every 10 seconds.
Which of the following represents the BEST solution for the given use-case?

- Create a high-resolution custom metric and push the data using a script triggered every 10 seconds
- Enable EC2 detailed monitoring
- Simply get it from the CloudWatch Metrics
- Open a support ticket with AWS

Overall explanation
Correct option:
Create a high-resolution custom metric and push the data using a script triggered every 10 seconds
Using high-resolution custom metric, your applications can publish metrics to CloudWatch with 1-second resolution. You can watch the metrics scroll across your screen seconds after they are published and you can set up high-resolution CloudWatch Alarms that evaluate as frequently as every 10 seconds. You can alert with High-Resolution Alarms, as frequently as 10-second periods. High-Resolution Alarms allow you to react and take actions faster and support the same actions available today with standard 1-minute alarms.



via - https://aws.amazon.com/blogs/aws/new-high-resolution-custom-metrics-and-alarms-for-amazon-cloudwatch/
Incorrect options:
Enable EC2 detailed monitoring - As part of basic monitoring, Amazon EC2 sends metric data to CloudWatch in 5-minute periods. To send metric data for your instance to CloudWatch in 1-minute periods, you can enable detailed monitoring on the instance, however, this comes at an additional cost.
Simply get it from the CloudWatch Metrics - You can get data from metrics. The basic monitoring data is available automatically in a 5-minute interval and detailed monitoring data is available in a 1-minute interval.
Open a support ticket with AWS - This option has been added as a distractor.

Question 18

A banking application needs to send real-time alerts and notifications based on any updates from the backend services. The company wants to avoid implementing complex polling mechanisms for these notifications.
Which of the following types of APIs supported by the Amazon API Gateway is the right fit?

- REST APIs
- WebSocket APIs
- HTTP APIs
- REST or HTTP APIs

Overall explanation
Correct option:
WebSocket APIs
In a WebSocket API, the client and the server can both send messages to each other at any time. Backend servers can easily push data to connected users and devices, avoiding the need to implement complex polling mechanisms.
For example, you could build a serverless application using an API Gateway WebSocket API and AWS Lambda to send and receive messages to and from individual users or groups of users in a chat room. Or you could invoke backend services such as AWS Lambda, Amazon Kinesis, or an HTTP endpoint based on message content.
You can use API Gateway WebSocket APIs to build secure, real-time communication applications without having to provision or manage any servers to manage connections or large-scale data exchanges. Targeted use cases include real-time applications such as the following:

1. Chat applications
2. Real-time dashboards such as stock tickers
3. Real-time alerts and notifications

API Gateway provides WebSocket API management functionality such as the following:

4. Monitoring and throttling of connections and messages
5. Using AWS X-Ray to trace messages as they travel through the APIs to backend services
6. Easy integration with HTTP/HTTPS endpoints

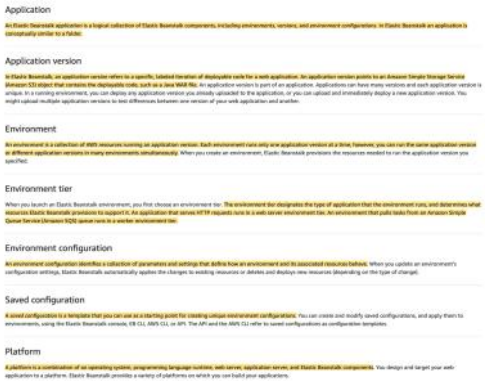
Incorrect options:
REST or HTTP APIs
REST APIs - An API Gateway REST API is made up of resources and methods. A resource is a logical entity that an app can access through a resource path. A method corresponds to a REST API request that is submitted by the user of your API and the response returned to the user.
For example, /incomes could be the path of a resource representing the income of the app user. A resource can have one or more operations that are defined by appropriate HTTP verbs such as GET, POST, PUT, PATCH, and DELETE. A combination of a resource path and an operation identifies a method of the API. For example, a POST /incomes method could add an income earned by the caller, and a GET /expenses method could query the reported expenses incurred by the caller.
HTTP APIs - HTTP APIs enable you to create RESTful APIs with lower latency and lower cost than REST APIs. You can use HTTP APIs to send requests to AWS Lambda functions or to any publicly routable HTTP endpoint.
For example, you can create an HTTP API that integrates with a Lambda function on the backend. When a client calls your API, API Gateway sends the request to the Lambda function and returns the function's response to the client.
Server push mechanism is not possible in REST and HTTP APIs.

Question 19

You company runs business logic on smaller software components that perform various functions. Some functions process information in a few seconds while others seem to take a long time to complete. Your manager asked you to decouple components that take a long time to ensure software applications stay responsive under load. You decide to configure Amazon Simple Queue Service (SQS) to work with your Elastic Beanstalk configuration.
Which of the following Elastic Beanstalk environment should you choose to meet this requirement?

- Single Instance with Elastic IP
- Dedicated worker environment
- Single Instance Worker node
- Load-balancing, Autoscaling environment

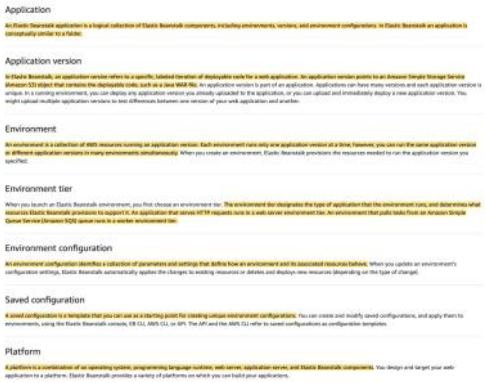
Overall explanation
Correct option:
With Elastic Beanstalk, you can quickly deploy and manage applications in the AWS Cloud without having to learn about the infrastructure that runs those applications. Elastic Beanstalk reduces management complexity without restricting choice or control. You simply upload your application, and Elastic Beanstalk automatically handles the details of capacity provisioning, load balancing, scaling, and application health monitoring.
Elastic BeanStalk Key Concepts:



via - https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/concepts.html

Dedicated worker environment - If your AWS Elastic Beanstalk application performs operations or workflows that take a long time to complete, you can offload those tasks to a dedicated worker environment. Decoupling your web application front end from a process that performs blocking operations is a common way to ensure that your application stays responsive under load.

A long-running task is anything that substantially increases the time it takes to complete a request, such as processing images or videos, sending emails, or generating a ZIP archive. These operations can take only a second or two to complete, but a delay of a few seconds is a lot for a web request that would otherwise complete in less than 500 ms.



via - https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/using-features-managing-env-tiers.html

Incorrect options:

Single Instance Worker node - Worker machines in Kubernetes are called nodes. Amazon EKS worker nodes are standard Amazon EC2 instances. EKS worker nodes are not to be confused with the Elastic Beanstalk worker environment. Since we are talking about the Elastic Beanstalk environment, this is not the correct choice.

Load-balancing, Autoscaling environment - A load-balancing and autoscaling environment uses the Elastic Load Balancing and Amazon EC2 Auto Scaling services to provision the Amazon EC2 instances that are required for your deployed application. Amazon EC2 Auto Scaling automatically starts additional instances to accommodate increasing load on your application. If your application requires scalability with the option of running in multiple Availability Zones, use a load-balancing, autoscaling environment. This is not the right environment for the given use-case since it will add costs to the overall solution.

Single Instance with Elastic IP - A single-instance environment contains one Amazon EC2 instance with an Elastic IP address. A single-instance environment doesn't have a load balancer, which can help you reduce costs compared to a load-balancing, autoscaling environment. This is not a highly available architecture, because if that one instance goes down then your application is down. This is not recommended for production environments.

Question 20

After reviewing your monthly AWS bill you notice that the cost of using Amazon SQS has gone up substantially after creating new queues; however, you know that your queue clients do not have a lot of traffic and are receiving empty responses.

Which of the following actions should you take?

- Use a FIFO queue
- Decrease DelaySeconds
- Use LongPolling
- Increase the VisibilityTimeout

Overall explanation

Correct option:

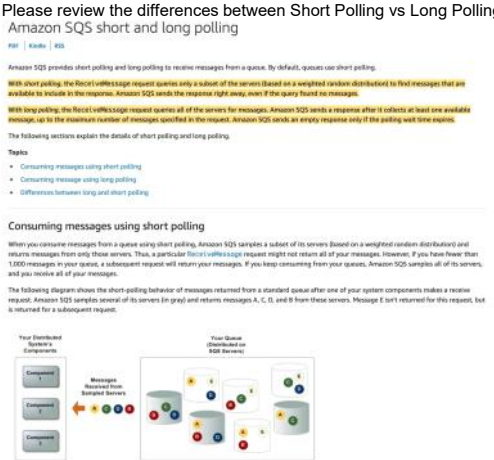
Use LongPolling

Amazon Simple Queue Service (SQS) is a fully managed message queuing service that enables you to decouple and scale microservices, distributed systems, and serverless applications. Amazon SQS provides short polling and long polling to receive messages from a queue. By default, queues use short polling. With short polling, Amazon SQS sends the response right away, even if the query found no messages. With long polling, Amazon SQS sends a response after it collects at least one available message, up to the maximum number of messages specified in the request. Amazon SQS sends an empty response only if the polling wait time expires.

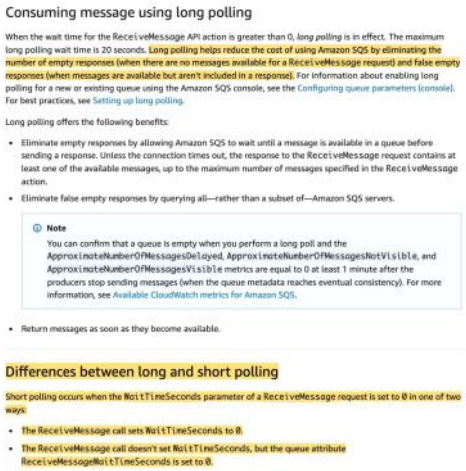
Long polling makes it inexpensive to retrieve messages from your Amazon SQS queue as soon as the messages are available. Long polling helps reduce the cost of using Amazon SQS by eliminating the number of empty responses (when there are no messages available for a ReceiveMessage request) and false empty responses (when messages are available but aren't included in a response). When the wait time for the ReceiveMessage API action is greater than 0, long polling is in effect. The maximum long polling wait time is 20 seconds.

Exam Alert:

Please review the differences between Short Polling vs Long Polling:



via - https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-short-and-long-polling.html



Consuming message using long polling

When the wait time for the ReceiveMessage API action is greater than 0, long polling is in effect. The maximum long polling wait time is 20 seconds. Long polling helps reduce the cost of using Amazon SQS by eliminating the number of empty responses (when there are no messages available for a ReceiveMessage request) and false empty responses (when messages are available but aren't included in a response). For information about enabling long polling for a new or existing queue using the Amazon SQS console, see the Configuring queue parameters (console). For best practices, see Setting up long polling.

Long polling offers the following benefits:

- Eliminate empty responses by allowing Amazon SQS to wait until a message is available in a queue before sending a response. Unless the connection times out, the response to the ReceiveMessage request contains at least one of the available messages, up to the maximum number of messages specified in the ReceiveMessage action.
- Eliminate false empty responses by querying all—rather than a subset of—Amazon SQS servers.

Note

You can confirm that a queue is empty when you perform a long poll and the ApproximateNumberOfMessagesDelayed, ApproximateNumberOfMessagesNotVisible, and ApproximateNumberOfMessagesVisible metrics are equal to 0 at least 1 minute after the producers stop sending messages (when the queue metadata reaches eventual consistency). For more information, see Available CloudWatch metrics for Amazon SQS.

- Return messages as soon as they become available.

Differences between long and short polling

Short polling occurs when the WaitTimeSeconds parameter of a ReceiveMessage request is set to 0 in one of two ways:

- The ReceiveMessage call sets WaitTimeSeconds to 0.
- The ReceiveMessage call doesn't set WaitTimeSeconds, but the queue attribute ReceiveMessageWaitTimeSeconds is set to 0.

via - https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-short-and-long-polling.html

Incorrect options:

Increase the VisibilityTimeout - Because there is no guarantee that a consumer received a message, the consumer must delete it. To prevent other consumers from processing the message again, Amazon SQS sets a visibility timeout. Visibility timeout will not help with cost reduction.

Use a FIFO queue - FIFO queues are designed to enhance messaging between applications when the order of operations and events has to be enforced . FIFO queues will not help with cost reduction. In fact, they are costlier than standard queues.

Decrease DelaySeconds - This is similar to VisibilityTimeout. The difference is that a message is hidden when it is first added to a queue for DelaySeconds, whereas for visibility timeouts a message is hidden only after it is consumed from the queue. DelaySeconds will not help with cost reduction.

Question 21

A company wants to add geospatial capabilities to the cache layer, along with query capabilities and an ability to horizontally scale. The company uses Amazon RDS as the database tier.
Which solution is optimal for this use-case?

- Leverage the capabilities offered by ElastiCache for Redis with cluster mode disabled
- Leverage the capabilities offered by ElastiCache for Redis with cluster mode enabled
- Use CloudFront caching to cater to demands of increasing workloads
- Migrate to Amazon DynamoDB to utilize the automatically integrated DynamoDB Accelerator (DAX) along with query capability features

Overall explanation

Correct option:

Leverage the capabilities offered by ElastiCache for Redis with cluster mode enabled

You can use Amazon ElastiCache to accelerate your high volume application workloads by caching your data in-memory providing sub-millisecond data retrieval performance. When used in conjunction with any database including Amazon RDS or Amazon DynamoDB, ElastiCache can alleviate the pressure associated with heavy request loads, increase overall application performance and reduce costs associated with scaling for throughput on other databases.

Amazon ElastiCache makes it easy to deploy and manage a highly available and scalable in-memory data store in the cloud. Among the open source in-memory engines available for use with ElastiCache is Redis, which added powerful geospatial capabilities in its newer versions.

You can leverage ElastiCache for Redis with cluster mode enabled to enhance reliability and availability with little change to your existing workload. Cluster Mode comes with the primary benefit of horizontal scaling up and down of your Redis cluster, with almost zero impact on the performance of the cluster.

Enabling Cluster Mode provides a number of additional benefits in scaling your cluster. In short, it allows you to scale in or out the number of shards (horizontal scaling) versus scaling up or down the node type (vertical scaling). This means that Cluster Mode can scale to very large amounts of storage (potentially 100s of terabytes) across up to 90 shards, whereas a single node can only store as much data in memory as the instance type has capacity for.

Cluster Mode also allows for more flexibility when designing new workloads with unknown storage requirements or heavy write activity. In a read-heavy workload, one can scale a single shard by adding read replicas, up to five, but a write-heavy workload can benefit from additional write endpoints when cluster mode is enabled.

Geospatial on Amazon ElastiCache for Redis:

via - https://aws.amazon.com/blogs/database/work-with-cluster-mode-on-amazon-elasticache-for-redis/

Incorrect options:

Leverage the capabilities offered by ElastiCache for Redis with cluster mode disabled - For a production workload, you should consider using a configuration that includes replication to enhance the protection of your data. Also, only vertical scaling is possible when cluster mode is disabled. The use case mentions horizontal scaling as a requirement, hence disabling cluster mode is not an option.

Use CloudFront caching to cater to demands of increasing workloads - One of the purposes of using CloudFront is to reduce the number of requests that your origin server must respond to directly. With CloudFront caching, more objects are served from CloudFront edge locations, which are closer to your users. This reduces the load on your origin server and reduces latency. However, the use case mentions that in-memory caching is needed for enhancing the performance of the application. So, this option is incorrect.

Migrate to Amazon DynamoDB to utilize the automatically integrated DynamoDB Accelerator (DAX) along with query capability features - Amazon DynamoDB Accelerator (DAX) is a fully managed, highly available, in-memory cache for DynamoDB that delivers up to a 10x performance improvement – from milliseconds to microseconds – even at millions of requests per second. Database migration is a more elaborate effort compared to implementing and optimizing the caching layer.

Question 22

A website serves static content from an Amazon Simple Storage Service (Amazon S3) bucket and dynamic content from an application load balancer. The user base is spread across the world and latency should be minimized for a better user experience.
Which technology/service can help access the static and dynamic content while keeping the data latency low?

- Configure CloudFront with multiple origins to serve both static and dynamic content at low latency to global users
- Use CloudFront's Origin Groups to group both static and dynamic requests into one request for further processing
- Use CloudFront's Lambda@Edge feature to server data from S3 buckets and load balancer programmatically on-the-fly
- Use Global Accelerator to transparently switch between S3 bucket and load balancer for different data needs

Overall explanation

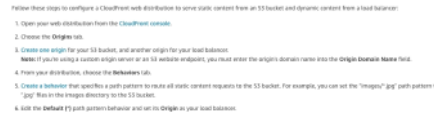
Correct option:

Configure CloudFront with multiple origins to serve both static and dynamic content at low latency to global users

Amazon CloudFront is a web service that speeds up distribution of your static and dynamic web content, such as .html, .css, .js, and image files, to your users. CloudFront delivers your content through a worldwide network of data centers called edge locations. When a user requests content that you're serving with CloudFront, the request is routed to the edge location that provides the lowest latency (time delay), so that content is delivered with the best possible performance.

You can configure a single CloudFront web distribution to serve different types of requests from multiple origins.

Steps to configure CLoudFront for multiple origins:



via - https://aws.amazon.com/premiumsupport/knowledge-center/cloudfront-distribution-serve-content/

Incorrect options:

Use CloudFront's Lambda@Edge feature to server data from S3 buckets and load balancer programmatically on-the-fly - AWS Lambda@Edge is a general-purpose serverless compute feature that supports a wide range of computing needs and customizations. Lambda@Edge is best suited for computationally intensive operations. This is not relevant for the given use case.

Use Global Accelerator to transparently switch between S3 bucket and load balancer for different data needs - AWS Global Accelerator is a networking service that improves the performance of your users' traffic by up to 60% using Amazon Web Services' global network infrastructure.

With Global Accelerator, you are provided two global static public IPs that act as a fixed entry point to your application, improving availability. On the back end, add or remove your AWS application endpoints, such as Application Load Balancers, Network Load Balancers, EC2 Instances, and Elastic IPs without making user-facing changes. Global Accelerator automatically re-routes your traffic to your nearest healthy available endpoint to mitigate endpoint failure.

CloudFront improves performance for both cacheable content (such as images and videos) and dynamic content (such as API acceleration and dynamic site delivery). Global Accelerator is a good fit for non-HTTP use cases, such as gaming (UDP), IoT (MQTT), or Voice over IP, as well as for HTTP use cases that specifically require static IP addresses or deterministic, fast regional failover. Global Accelerator is not relevant for the given use-case.

Use CloudFront's Origin Groups to group both static and dynamic requests into one request for further processing - You can set up CloudFront with origin failover for scenarios that require high availability. To get started, you create an Origin Group with two origins: a primary and a secondary. If the primary origin is unavailable or returns specific HTTP response status codes that indicate a failure, CloudFront automatically switches to the secondary origin. Origin Groups are for origin failure scenarios and not for request routing.

Question 23

An order management system uses a cron job to poll for any new orders. Every time a new order is created, the cron job sends this order data as a message to the message queues to facilitate downstream order processing in a reliable way. To reduce costs and improve performance, the company wants to move this functionality to AWS cloud.

Which of the following is the most optimal solution to meet this requirement?

- Use Amazon Simple Notification Service (SNS) to push notifications when an order is created. Configure different Amazon Simple Queue Service (SQS) queues to receive these messages for downstream processing
- Use Amazon Simple Notification Service (SNS) to push notifications and use AWS Lambda functions to process the information received from SNS
- Use Amazon Simple Notification Service (SNS) to push notifications to Kinesis Data Firehose delivery streams for processing the data for downstream applications
- Configure different Amazon Simple Queue Service (SQS) queues to poll for new orders

Overall explanation

Correct option:

Use Amazon Simple Notification Service (SNS) to push notifications when an order is created. Configure different Amazon Simple Queue Service (SQS) queues to receive these messages for downstream processing

Amazon SNS works closely with Amazon Simple Queue Service (Amazon SQS). These services provide different benefits for developers. Amazon SNS allows applications to send time-critical messages to multiple subscribers through a "push" mechanism, eliminating the need to periodically check or "poll" for updates. Amazon SQS is a message queue service used by distributed applications to exchange messages through a polling model, and can be used to decouple sending and receiving components—without requiring each component to be concurrently available.

Using Amazon SNS and Amazon SQS together, messages can be delivered to applications that require immediate notification of an event, and also stored in an Amazon SQS queue for other applications to process at a later time.

When you subscribe an Amazon SQS queue to an Amazon SNS topic, you can publish a message to the topic and Amazon SNS sends an Amazon SQS message to the subscribed queue. The Amazon SQS message contains the subject and message that were published to the topic along with metadata about the message in a JSON document.

SNS-SQS fanout is the right solution for this use case.

Sample SNS-SQS Fanout message:

via - https://docs.aws.amazon.com/sns/latest/dg/sns-sqs-as-subscriber.html

Incorrect options:

Configure different Amazon Simple Queue Service (SQS) queues to poll for new orders - Amazon SQS cannot be used as a polling service, as messages need to be pushed to the queue, which are then handled by the queue consumers.

Use Amazon Simple Notification Service (SNS) to push notifications and use AWS Lambda functions to process the information received from SNS - Amazon SNS and AWS Lambda are integrated so you can invoke Lambda functions with Amazon SNS notifications. When a message is published to an SNS topic that has a Lambda function subscribed to it, the Lambda function is invoked with the payload of the published message. For the given scenario, we need a service that can store the message data pushed by SNS, for further processing. AWS Lambda does not have capacity to store the message data. In case a Lambda function is unable to process a specific message, it will be left unprocessed. Hence this option is not correct.

Use Amazon Simple Notification Service (SNS) to push notifications to Kinesis Data Firehose delivery streams for processing the data for downstream applications - You can subscribe Amazon Kinesis Data Firehose delivery streams to SNS topics, which allows you to send notifications to additional storage and analytics endpoints. However, Kinesis is built for real-time processing of big data. Whereas, SQS is meant for decoupling dependent systems with easy methods to transmit data/messages. SQS also is a cheaper option when compared to Firehose. Therefore this option is not the right fit for the given use case.

Question 24

An organization uses Alexa as its intelligent assistant to improve productivity throughout the organization. A group of developers manages custom Alexa Skills written in Node.Js to control conference-room equipment settings and start meetings using voice activation. The manager has requested developers that all functions code should be monitored for error rates with the possibility of creating alarms on top of them.

Which of the following options should be chosen? (select two)

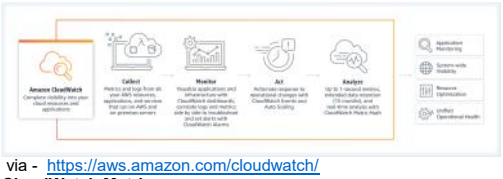
- SSM
- CloudWatch Metrics
- CloudTrail
- CloudWatch Alarms
- X-Ray

Overall explanation

Correct options:

CloudWatch collects monitoring and operational data in the form of logs, metrics, and events, and visualizes it using automated dashboards so you can get a unified view of your AWS resources, applications, and services that run in AWS and on-premises. You can correlate your metrics and logs to better understand the health and performance of your resources. You can also create alarms based on metric value thresholds you specify, or that can watch for anomalous metric behavior based on machine learning algorithms.

How CloudWatch works:



via - https://aws.amazon.com/cloudwatch/

CloudWatch Metrics

Amazon CloudWatch monitors your Amazon Web Services (AWS) resources and the applications you run on AWS in real-time. You can use CloudWatch to collect and track metrics, which are variables you can measure for your resources and applications. Metric data is kept for 15 months, enabling you to view both up-to-the-minute data and historical data.

CloudWatch retains metric data as follows:

Data points with a period of less than 60 seconds are available for 3 hours. These data points are high-resolution custom metrics. Data points with a period of 60 seconds (1 minute) are available for 15 days Data points with a period of 300 seconds (5 minute) are available for 63 days Data points with a period of 3600 seconds (1 hour) are available for 455 days (15 months)

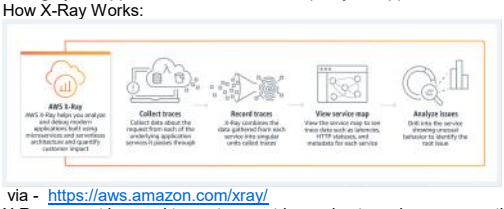
CloudWatch Alarms

You can use an alarm to automatically initiate actions on your behalf. An alarm watches a single metric over a specified time, and performs one or more specified actions, based on the value of the metric relative to a threshold over time. The action is a notification sent to an Amazon SNS topic or an Auto Scaling policy. You can also add alarms to dashboards.

CloudWatch alarms send notifications or automatically make changes to the resources you are monitoring based on rules that you define. Alarms work together with CloudWatch Metrics.

A metric alarm has the following possible states:

OK – The metric or expression is within the defined threshold.
ALARM – The metric or expression is outside of the defined threshold.
INSUFFICIENT_DATA – The alarm has just started, the metric is not available, or not enough data is available for the metric to determine the alarm state.
Incorrect options:
X-Ray - AWS X-Ray helps developers analyze and debug production, distributed applications, such as those built using a microservices architecture. With X-Ray, you can understand how your application and its underlying services are performing to identify and troubleshoot the root cause of performance issues and errors. X-Ray provides an end-to-end view of requests as they travel through your application, and shows a map of your application's underlying components.
How X-Ray Works:



via - https://aws.amazon.com/xray/
X-Ray cannot be used to capture metrics and set up alarms as per the given use-case, so this option is incorrect.
CloudTrail - CloudWatch is a monitoring service whereas CloudTrail is more of an audit service where you can find API calls made on services and by whom.
How CloudTrail Works:

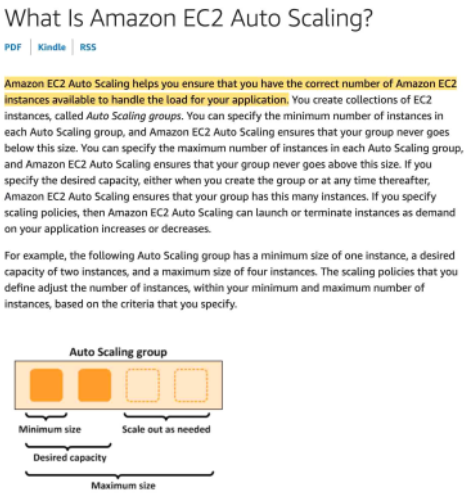
via - https://aws.amazon.com/cloudtrail/
Systems Manager - Using AWS Systems Manager, you can group resources, like Amazon EC2 instances, Amazon S3 buckets, or Amazon RDS instances, by application, view operational data for monitoring and troubleshooting, and take action on your groups of resources. Systems Manager cannot be used to capture metrics and set up alarms as per the given use-case, so this option is incorrect.

Question 25

A data analytics company ingests a large number of messages and stores them in an RDS database using Lambda. Because of the increased payload size, it is taking more than 15 minutes to process each message.
As a Developer Associate, which of the following options would you recommend to process each message in the MOST scalable way?

- Provision an EC2 instance to poll the messages from an SQS queue
- Contact AWS Support to increase the Lambda timeout to 60 minutes
- Provision EC2 instances in an Auto Scaling group to poll the messages from an SQS queue
- Use DynamoDB instead of RDS as database

Overall explanation
Correct option:
Provision EC2 instances in an Auto Scaling group to poll the messages from an SQS queue
As each message takes more than 15 minutes to process, therefore the development team cannot use Lambda for message processing. To build a scalable solution, the EC2 instances must be provisioning via an Auto Scaling group to handle variations in the message processing workload.
Amazon EC2 Auto Scaling Overview:



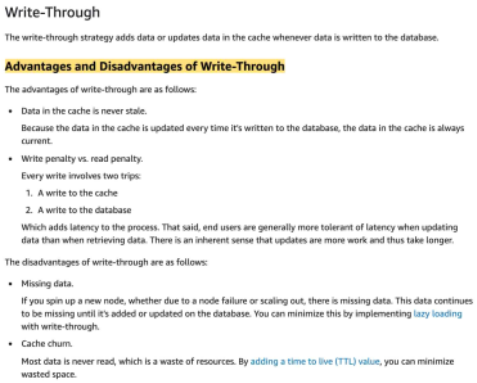
via - https://docs.aws.amazon.com/autoscaling/ec2/userguide/what-is-amazon-ec2-auto-scaling.html
Incorrect options:
Provision an EC2 instance to poll the messages from an SQS queue - Just using a single EC2 instance may not be sufficient to handle a sudden spike in the number of incoming messages.
Contact AWS Support to increase the Lambda timeout to 60 minutes - AWS Support cannot increase the Lambda timeout upper limit.
Use DynamoDB instead of RDS as database - This option has been added as a distractor, as changing the database would have no impact on the Lambda timeout while processing the message.

Question 26

You are running a web application where users can author blogs and share them with their followers. Most of the workflow is read based, but when a blog is updated, you would like to ensure that the latest data is served to the users (no stale data). The Developer has already suggested using ElastiCache to cope with the read load but has asked you to implement a caching strategy that complies with the requirements of the site.
Which strategy would you recommend?

Use a Lazy Loading strategy without TTL
Use DAX
Use a Lazy Loading strategy with TTL
Use a Write Through strategy

Overall explanation
Correct option:
Use a Write Through strategy
The write-through strategy adds data or updates data in the cache whenever data is written to the database.
In a Write Through strategy, any new blog or update to the blog will be written to both the database layer and the caching layer, thus ensuring that the latest data is always served from the cache.



Write-Through

The write-through strategy adds data or updates data in the cache whenever data is written to the database.

Advantages and Disadvantages of Write-Through

The advantages of write-through are as follows:

- Data in the cache is never stale.
  Because the data in the cache is updated every time it's written to the database, the data in the cache is always current.
- Write penalty vs. read penalty.
  Every write involves two trips:
  1. A write to the cache
  2. A write to the database
  Which adds latency to the process. That said, end users are generally more tolerant of latency when updating data than when retrieving data. There is an inherent sense that updates are more work and thus take longer.

The disadvantages of write-through are as follows:

- Missing data.
  If you spin up a new node, whether due to a node failure or scaling out, there is missing data. This data continues to be missing until it's added or updated on the database. You can minimize this by implementing lazy loading with write-through.
- Cache churn.
  Most data is never read, which is a waste of resources. By adding a time to live (TTL) value, you can minimize wasted space.

via - https://docs.aws.amazon.com/AmazonElastiCache/latest/mem-ug/Strategies.html

Incorrect options:

Use a Lazy Loading strategy without TTL

Lazy Loading is a caching strategy that loads data into the cache only when necessary. Whenever your application requests data, it first requests the ElastiCache cache. If the data exists in the cache and is current, ElastiCache returns the data to your application. If the data doesn't exist in the cache or has expired, your application requests the data from your data store.

Use a Lazy Loading strategy with TTL

In the case of Lazy Loading, the data is loaded onto the cache whenever the data is missing from the cache. In case the blog gets updated, it won't be updated from the cache unless that cache expires (in case you used a TTL). Time to live (TTL) is an integer value that specifies the number of seconds until the key expires. When an application attempts to read an expired key, it is treated as though the key is not found. The database is queried for the key and the cache is updated. Therefore, for a while, old data will be served to users which is a problem from a requirements perspective as we don't want any stale data.

Use DAX - This is a cache for DynamoDB based implementations, but in this question, we are considering ElastiCache. Therefore this option is not relevant.

Question 27

You are working with a t2.small instance bastion host that has the AWS CLI installed to help manage all the AWS services installed on it. You would like to know the security group and the instance id of the current instance.
Which of the following will help you fetch the needed information?

- Query the user data at http://254.169.254.169/latest/meta-data
- Create an IAM role and attach it to your EC2 instance that helps you perform a 'describe' API call
- Query the user data at http://169.254.169.254/latest/user-data
- Query the metadata at http://169.254.169.254/latest/meta-data

Overall explanation

Correct option:

Query the metadata at http://169.254.169.254/latest/meta-data - Because your instance metadata is available from your running instance, you do not need to use the Amazon EC2 console or the AWS CLI. This can be helpful when you're writing scripts to run from your instance. For example, you can access the local IP address of your instance from instance metadata to manage a connection to an external application. To view all categories of instance metadata from within a running instance, use the following URI - http://169.254.169.254/latest/meta-data/. The IP address 169.254.169.254 is a link-local address and is valid only from the instance. All instance metadata is returned as text (HTTP content type text/plain).

Incorrect options:

Create an IAM role and attach it to your EC2 instance that helps you perform a 'describe' API call - The AWS CLI has a describe-instances API call needs instance ID as an input. So, this will not work for the current use case wherein we do not know the instance ID.

Query the user data at http://169.254.169.254/latest/user-data - This address retrieves the user data that you specified when launching your instance.

Query the user data at http://254.169.254.169/latest/meta-data - The IP address specified is wrong.

References:

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/instancedata-data-retrieval.html
https://awscli.amazonaws.com/v2/documentation/api/latest/reference/ec2/describe-instances.html

Question 28

A business-critical application is hosted on an Amazon EC2 instance and the latest update is in the testing phase. The business has requested a solution to track the average response time of the application and send a notification to the manager if it exceeds a particular threshold. The update will eventually be implemented in the production environment where the solution will be deployed on multiple EC2 instances.
Which of the following options would you combine to address the requirements of the business? (Select two)

- Configure the application to write the response time to a log file on the instance. Install and configure the Amazon Inspector agent on the EC2 instances to read the logs and send the response time to Amazon EventBridge
- Configure an EventBridge custom rule to send an Amazon Simple Notification Service (Amazon SNS) notification when the average of the response time metric exceeds the threshold
- Install and configure AWS Systems Manager Agent (SSM Agent) on the EC2 instances to monitor the response time and send the data to Amazon CloudWatch as a custom metric
- Create a CloudWatch alarm to send an Amazon Simple Notification Service (Amazon SNS) notification when the average of the response time metric exceeds the threshold
- Configure the application to write the response time to a log file on the EC2 instance. Install and configure the Amazon CloudWatch agent on the EC2 instance to stream the application logs to CloudWatch Logs. Create a metric filter for the response time from the log file

Overall explanation

Correct options:

Configure the application to write the response time to a log file on the EC2 instance. Install and configure the Amazon CloudWatch agent on the EC2 instance to stream the application logs to CloudWatch Logs. Create a metric filter for the response time from the log file

Create a CloudWatch alarm to send an Amazon Simple Notification Service (Amazon SNS) notification when the average of the response time metric exceeds the threshold

You can collect metrics and logs from Amazon EC2 instances and on-premises servers with the CloudWatch agent. The unified CloudWatch agent enables you to do the following:

7. Collect internal system-level metrics from Amazon EC2 instances across operating systems. The metrics can include in-guest metrics, in addition to the metrics for EC2 instances.
8. Collect system-level metrics from on-premises servers. These can include servers in a hybrid environment as well as servers not managed by AWS.
9. Retrieve custom metrics from your applications or services using the StatsD and collectd protocols. StatsD is supported on both Linux servers and servers running Windows Server. collectd is supported only on Linux servers.
10. Collect logs from Amazon EC2 instances and on-premises servers, running either Linux or Windows Server.

You can store and view the metrics that you collect with the CloudWatch agent in CloudWatch just as you can with any other CloudWatch metrics. The default namespace for metrics collected by the CloudWatch agent is CWAgent, although you can specify a different namespace when you configure the agent.

A metric alarm watches a single CloudWatch metric or the result of a math expression based on CloudWatch metrics. The alarm performs one or more actions based on the value of the metric or expression relative to a threshold over a number of time periods. The action can be sending a notification to an Amazon SNS topic, performing an Amazon EC2 action or an Amazon EC2 Auto Scaling action, or creating an OpsItem or incident in Systems Manager. For this use case, we will configure the metric alarm action to send an SNS notification when the alarm is in an ALARM state.

Incorrect options:

Configure the application to write the response time to a log file on the instance. Install and configure the Amazon Inspector agent on the EC2 instances to read the logs and send the response time to Amazon EventBridge - This statement is incorrect. Amazon Inspector is an automated vulnerability management service that continually scans Amazon Elastic Compute Cloud (EC2), AWS Lambda functions, and container workloads for software vulnerabilities and unintended network exposure. Inspector is not a log-collecting agent.

Configure an EventBridge custom rule to send an Amazon Simple Notification Service (Amazon SNS) notification when the average of the response time metric exceeds the threshold - Amazon EventBridge integrates with Amazon CloudWatch so that when CloudWatch alarms are triggered, a matching EventBridge rule can execute targets. So we are using CloudWatch alarms in this scenario too and hence a straightforward way would be to use CloudWatch alarm.

Install and configure AWS Systems Manager Agent (SSM Agent) on the EC2 instances to monitor the response time and send the data to Amazon CloudWatch as a custom metric - This statement is incorrect. AWS Systems Manager Agent (SSM Agent) is Amazon software that runs on Amazon Elastic Compute Cloud (Amazon EC2) instances, edge devices, on-premises servers, and virtual machines (VMs). SSM Agent makes it possible for the Systems Manager to update, manage, and configure these resources. SSM agent is required on each instance that needs to be managed from the Systems Manager service. SSM agents cannot collect log data and push it to CloudWatch logs. However, Systems Manager can be used to automate the task of CloudWatch agent installation on the instances.

Question 29

You would like to paginate the results of an S3 List to show 100 results per page to your users and minimize the number of API calls that you will use.
Which CLI options should you use? (Select two)

- --next-token

- --page-size
- --max-items
- --limit
- --starting-token

Overall explanation
Correct options:
--max-items
--starting-token
For commands that can return a large list of items, the AWS Command Line Interface (AWS CLI) has three options to control the number of items included in the output when the AWS CLI calls a service's API to populate the list.
--page-size
--max-items
--starting-token
By default, the AWS CLI uses a page size of 1000 and retrieves all available items. For example, if you run aws s3api list-objects on an Amazon S3 bucket that contains 3,500 objects, the AWS CLI makes four calls to Amazon S3, handling the service-specific pagination logic for you in the background and returning all 3,500 objects in the final output.
Here's an example: aws s3api list-objects --bucket my-bucket --max-items 100 --starting-token eyJNYXJrZXIiOiBudWxsLCAiYm90b190cnVuY2F0ZV9hbW91bnQiOiAxfQ==
Incorrect options:
"--page-size" - You can use the --page-size option to specify that the AWS CLI requests a smaller number of items from each call to the AWS service. The CLI still retrieves the full list but performs a larger number of service API calls in the background and retrieves a smaller number of items with each call.
"--next-token" - This is a made-up option and has been added as a distractor.
"--limit" - This is a made-up option and has been added as a distractor.

Question 30

You are running a public DNS service on an EC2 instance where the DNS name is pointing to the IP address of the instance. You wish to upgrade your DNS service but would like to do it without any downtime.
Which of the following options will help you accomplish this?

- Create a Load Balancer and an auto scaling group
- Use Route 53
- Elastic IP
- Provide a static private IP

Overall explanation
Correct option:
Route 53 is a DNS managed by AWS, but nothing prevents you from running your own DNS (it's just a software) on an EC2 instance. The trick of this question is that it's about EC2, running some software that needs a fixed IP, and not about Route 53 at all.
Elastic IP
DNS services are identified by a public IP, so you need to use Elastic IP.
Incorrect options:
Create a Load Balancer and an auto-scaling group - Load balancers do not provide an IP, instead they provide a DNS name, so this option is ruled out.
Provide a static private IP - If you provide a private IP it will not be accessible from the internet, so this option is incorrect.
Use Route 53 - Route 53 is a DNS service from AWS but the use-case talks about offering a DNS service using an EC2 instance, so this option is incorrect.
Reference:
https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/elastic-ip-addresses-eip.html#using-instance-addressing-eips-associating-different

Question 31

A developer has defined a Lambda integration in Amazon API Gateway using a stage variable. However, when the developer invokes the API method, it consistently returns an "Internal server error" and a 500 status code.
What steps should the developer take to fix the issue?

- API calls can't exceed the maximum allowed API request rate per account and per Region. Implement error retries and exponential backoffs to fix the error
- If you create a stage variable to call a Lambda function through your API, you must add the required permissions. Update your Lambda function's resource-based AWS Identity and Access Management (IAM) policy so that it grants invoke permission to the API Gateway
- If you create a stage variable to call a function through your API, you must add the required permissions. Create an IAM role that your Lambda function can assume when invoking the respective AWS resources
- When setting the Lambda function as the value of a stage variable, use the function's ARN and not the function alias for setting up the value

Overall explanation
Correct option:
If you create a stage variable to call a Lambda function through your API, you must add the required permissions. Update your Lambda function's resource-based AWS Identity and Access Management (IAM) policy so that it grants invoke permission to the API Gateway
If your Lambda function's resource-based policy doesn't include permissions for your API to invoke the function, API Gateway returns an Internal server error message.
If you create a stage variable to call a function through your API, you must add the required permissions by doing one of the following:
Update your Lambda function's resource-based AWS Identity and Access Management (IAM) policy so that it grants invoke permission to API Gateway.
OR
Create an IAM role that API Gateway can assume to invoke your Lambda function.
If you build an API Gateway API with standard Lambda integration using the API Gateway console, the console adds the required permissions automatically.
Example resource-based policy that grants invoke permission to API Gateway:

via - https://aws.amazon.com/premiumsupport/knowledge-center/api-gateway-lambda-stage-variable-500/
Incorrect options:
API calls can't exceed the maximum allowed API request rate per account and per Region. Implement error retries and exponential backoffs to fix the error - When API calls exceed the maximum allowed API request rate per account and per Region, the error received is a "Rate Exceeded" error and not an "Internal server error". Hence, this option is irrelevant to the given use case.
If you create a stage variable to call a function through your API, you must add the required permissions. Create an IAM role that your Lambda function can assume when invoking the respective AWS resources - This option is incorrect. However, creating an IAM role that API Gateway can assume to invoke the Lambda function is another solution to fix the given problem.
When setting the Lambda function as the value of a stage variable, use the function's ARN and not the function alias for setting up the value - This statement is incorrect. When setting a Lambda function as the value of a stage variable, use the function's local name, possibly including its alias or version specification. Do not use the function's ARN. The API Gateway console assumes the stage variable value for a Lambda function as the unqualified function name and expands the given stage variable into an ARN.
References:
https://docs.aws.amazon.com/lambda/latest/dg/services-apigateway.html
https://docs.aws.amazon.com/apigateway/latest/developerguide/how-to-set-stage-variables-aws-console.html
https://aws.amazon.com/premiumsupport/knowledge-center/api-gateway-lambda-stage-variable-500/
Domain
Troubleshooting and Optimization

From <https://www.udemy.com/course/aws-certified-developer-associate-practice-tests-dva-c01/learn/quiz/4682824/result/1739504465?expanded=1739504465#overview>

Question 32

A developer has just completed configuring the Application Load Balancer for the EC2 instances. Just as he started testing his configuration, he realized that he has missed assigning target groups to his ALB.
Which error code should he expect in his debug logs?

- HTTP 504
- HTTP 500
- HTTP 403
- HTTP 503

Overall explanation
Correct option:
HTTP 503 - HTTP 503 indicates 'Service unavailable' error. This error in ALB is an indicator of the target groups for the load balancer having no registered targets.
Incorrect options:
HTTP 500 - HTTP 500 indicates 'Internal server' error. There are several reasons for their error: A client submitted a request without an HTTP protocol, and the load balancer was unable to generate a redirect URL, there was an error executing the web ACL rules.
HTTP 504 - HTTP 504 is 'Gateway timeout' error. Several reasons for this error, to quote a few: The load balancer failed to establish a connection to the target before the connection timeout expired, The load balancer established a connection to the target but the target did not respond before the idle timeout period elapsed.
HTTP 403 - HTTP 403 is 'Forbidden' error. You configured an AWS WAF web access control list (web ACL) to monitor requests to your Application Load Balancer and it blocked a request.
Reference:
https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-troubleshooting.html

Question 33

A developer is configuring Amazon ECS container instances to send log information to CloudWatch Logs. For the container instances to be able to send log data to CloudWatch Logs, an IAM policy needs to be created that will allow the container instances to use the CloudWatch Logs APIs.
Which policy is the right fit for the given requirement?

```
{
   "Version": "2012-10-17",
   "Statement": [
      {
         "Effect": "Allow",
         "Action": [
            "logs:CreateLogGroup",
            "logs:CreateLogStream",
            "logs:PutLogEvents",
            "logs:DescribeLogGroups"
         ],
         "Resource": [
            "arn:aws:logs:*:*:*"
         ]
      }
   ]
}
```

Correct answer

```
{
   "Version": "2012-10-17",
   "Statement": [
      {
         "Effect": "Allow",
         "Action": [
            "logs:CreateLogGroup",
            "logs:CreateLogStream",
            "logs:PutLogEvents",
            "logs:DescribeLogStreams"
         ],
         "Resource": [
            "arn:aws:logs:*:*:*"
         ]
      }
   ]
}
```

```
{
   "Version": "2012-10-17",
   "Statement": [
      {
         "Effect": "Allow",
         "Action": [
            "logs:CreateLogGroup",
            "logs:CreateLogStream",
            "logs:PutLogEvents"
         ],
         "Resource": [
            "arn:aws:logs:*:*:*"
         ]
      }
   ]
}
```

Your answer is incorrect

```
{
   "Version": "2012-10-17",
   "Statement": [
      {
         "Effect": "Allow",
         "Action": [
            "logs:CreateLogGroup",
            "logs:CreateLogStream",
            "logs:PutLogEvents,
            "ecs:DescribeServices"
         ],
         "Resource": [
            "arn:aws:logs:<ARN of the Log Group>"
         ]
      }
   ]
}
```

Overall explanation
Correct option:

```
{
   "Version": "2012-10-17",
   "Statement": [
      {
         "Effect": "Allow",
         "Action": [
            "logs:CreateLogGroup",
            "logs:CreateLogStream",
```

```
                "logs:PutLogEvents",
                "logs:DescribeLogStreams"
            ],
            "Resource": [
                "arn:aws:logs:*:*:*"
            ]
        }
    ]
}
```

Before your container instances can send log data to CloudWatch Logs, you must create an IAM policy to allow your container instances to use the CloudWatch Logs APIs, and then you must attach that policy to ecsInstanceRole.
This policy has one statement that grants permissions to create log groups and log streams, to upload log events to log streams, and to list details about log streams.
The wildcard character *(*) at the end of the Resource value means that the statement allows permission for the logs:CreateLogGroup, logs:CreateLogStream, logs:PutLogEvents, and logs:DescribeLogStreams actions on any log group. To limit this permission to a specific log group, replace the wildcard character (*)* in the resource ARN with the specific log group ARN
Incorrect options:

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "logs:CreateLogGroup",
                "logs:CreateLogStream",
                "logs:PutLogEvents"
            ],
            "Resource": [
                "arn:aws:logs:*:*:*"
            ]
        }
    ]
}
```

- Permission to list details of the log stream needs to be attached to this policy.

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "logs:CreateLogGroup",
                "logs:CreateLogStream",
                "logs:PutLogEvents,
                "ecs:DescribeServices"
            ],
            "Resource": [
                "arn:aws:logs:<ARN of the Log Group>"
            ]
        }
    ]
}```
```

- ecs:DescribeServices permission is not needed, but logs:DescribeLogStreams permissions are needed for the policy to perform as expected.

```
{ "Version": "2012-10-17", "Statement": [ { "Effect": "Allow", "Action": [ "logs:CreateLogGroup", "logs:CreateLogStream", "logs:PutLogEvents", "logs:DescribeLogGroups" ], "Resource": [ "arn:aws:logs:::*" ] } ] } ```
```

- logs:DescribeLogGroups is an erroneous permission here.

Question 34

A multi-national company runs its technology operations on AWS Cloud. As part of their storage solution, they use a large number of EBS volumes, with AWS Config and CloudTrail activated. A manager has tried to find the user name that created an EBS volume by searching CloudTrail events logs but wasn't successful.
As a Developer Associate, which of the following would you recommend as the correct solution?

- AWS CloudTrail event logs for 'ManageVolume' aren't available for EBS volumes created during an Amazon EC2 launch
- AWS CloudTrail event logs for 'CreateVolume' aren't available for EBS volumes created during an Amazon EC2 launch
- EBS volume status checks are disabled
- Amazon EBS CloudWatch metrics are disabled

Overall explanation
Correct option:
AWS CloudTrail event logs for 'CreateVolume' aren't available for EBS volumes created during an Amazon EC2 launch - AWS CloudTrail event logs for 'CreateVolume' aren't available for EBS volumes created during an Amazon Elastic Compute Cloud (Amazon EC2) launch.
Incorrect options:
AWS CloudTrail event logs for 'ManageVolume' aren't available for EBS volumes created during an Amazon EC2 launch - Event 'ManageVolume' is a made-up option and has been added as a distractor.
Amazon EBS CloudWatch metrics are disabled - Amazon Elastic Block Store (Amazon EBS) sends data points to CloudWatch for several metrics. Data is only reported to CloudWatch when the volume is attached to an instance. CloudWatch metrics are useful in tracking the status or life cycle changes of an EBS volume, they are not useful in knowing about the metadata of EBS volumes.
EBS volume status checks are disabled - Volume status checks enable you to better understand, track and manage potential inconsistencies in the data on an Amazon EBS volume. They are designed to provide you with the information that you need to determine whether your Amazon EBS volumes are impaired, and to help you control how a potentially inconsistent volume is handled. Our current use case requires us to pull data about EBS volume metadata, which is not possible with this feature.

Question 35

A startup manages its Cloud resources with Elastic Beanstalk. The environment consists of few Amazon EC2 instances, an Auto Scaling Group (ASG), and an Elastic Load Balancer. Even after the Load Balancer marked an EC2 instance as unhealthy, the ASG has not replaced it with a healthy instance.
As a Developer, suggest the necessary configurations to automate the replacement of unhealthy instance.

- The health check type of your instance's Auto Scaling group, must be changed from EC2 to ELB by using a configuration file
- The ping path field of the Load Balancer is configured incorrectly
- Health check parameters were configured for checking the instance health alone. The instance failed because of application failure which was not configured as a parameter for health check status
- Auto Scaling group doesn't automatically replace the unhealthy instances marked by the load balancer. They have to be manually replaced from AWS Console

Overall explanation
Correct option:
The health check type of your instance's Auto Scaling group, must be changed from EC2 to ELB by using a configuration file - By default, the health check configuration of your Auto Scaling group is set as an EC2 type that performs a status check of EC2 instances. To automate the replacement of unhealthy EC2 instances, you must change the health check type of your instance's Auto Scaling group from EC2 to ELB by using a configuration file.
Incorrect options:
Health check parameters were configured for checking the instance health alone. The instance failed because of application failure which was not configured as a parameter for health check status - This is an incorrect statement. Status checks, by definition, cover only an EC2 instance's health, and not the health of your application, server, or any Docker containers running on the instance.
Auto Scaling group doesn't automatically replace the unhealthy instances marked by the load balancer. They have to be manually replaced from AWS Console - Incorrect statement. As discussed above, if the health check type of ASG is changed from EC2 to ELB, Auto Scaling will be able to replace the unhealthy instance.
The ping path field of the Load Balancer is configured incorrectly - Ping path is a health check configuration field of Elastic Load Balancer. If the ping path is configured wrong, ELB will not be able to reach the instance and hence will consider the instance unhealthy. However, this would then apply to all instances, not just once instance. So it does not address the issue given in the use-case.

Question 36

A company wants to implement authentication for its new RESTful API service that uses Amazon API Gateway. To authenticate the calls, each request must include HTTP headers with a client ID and user ID. These credentials must be compared to the authentication data in a DynamoDB table.

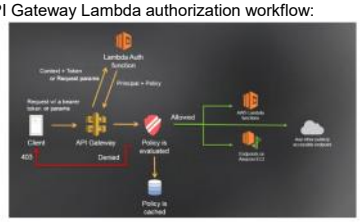
As an AWS Certified Developer Associate, which of the following would you recommend for implementing this authentication in API Gateway?

- Authorize using Amazon Cognito that will reference the authentication table of DynamoDB
- Update the API Gateway integration requests to require the credentials, then grant API Gateway access to the authentication table in DynamoDB
- Develop an AWS Lambda authorizer that references the authentication data in the DynamoDB table
- Set up an API Gateway Model that requires the credentials, then grant API Gateway access to the authentication table in DynamoDB

Overall explanation
Correct option:
Develop an AWS Lambda authorizer that references the DynamoDB authentication table - A Lambda authorizer is an API Gateway feature that uses a Lambda function to control access to your API.
A Lambda authorizer is useful if you want to implement a custom authorization scheme that uses a bearer token authentication strategy such as OAuth or SAML, or that uses request parameters to determine the caller's identity.
When a client makes a request to one of your API's methods, API Gateway calls your Lambda authorizer, which takes the caller's identity as input and returns an IAM policy as output.
There are two types of Lambda authorizers:

11. A token-based Lambda authorizer (also called a TOKEN authorizer) receives the caller's identity in a bearer token, such as a JSON Web Token (JWT) or an OAuth token.
12. A request parameter-based Lambda authorizer (also called a REQUEST authorizer) receives the caller's identity in a combination of headers, query string parameters, state variables, and $context variables.

API Gateway Lambda authorization workflow:



API Gateway Lambda authorization workflow

1. The client calls a method on an API Gateway API method, passing a bearer token or request parameters.
2. API Gateway checks whether a Lambda authorizer is configured for the method. If it is, API Gateway calls the Lambda function.
3. The Lambda function authenticates the caller by means such as the following:
   - Calling out to an OAuth provider to get an OAuth access token.
   - Calling out to a SAML provider to get a SAML assertion.
   - Generating an IAM policy based on the request parameter values.
   - Retrieving credentials from a database.
4. If the call succeeds, the Lambda function grants access by returning an output object containing at least an IAM policy and a principal identifier.
5. API Gateway evaluates the policy.
   - If access is denied, API Gateway returns a suitable HTTP status code, such as 403 ACCESS_DENIED.
   - If access is allowed, API Gateway executes the method. If caching is enabled in the authorizer settings, API Gateway also caches the policy so that the Lambda authorizer function doesn't need to be invoked again.

via - https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-use-lambda-authorizer.html
Incorrect options:
Set up an API Gateway Model that requires the credentials, then grant API Gateway access to the authentication table in DynamoDB - In API Gateway, a model defines the data structure of a payload. In API Gateway models are defined using the JSON schema draft 4. Models are not mandatory.
Update the API Gateway integration requests to require the credentials, then grant API Gateway access to the authentication table in DynamoDB - After setting up an API method, you must integrate it with an endpoint in the backend. A backend endpoint is also referred to as an integration endpoint and can be a Lambda function, an HTTP webpage, or an AWS service action. An integration request is an HTTP request that API Gateway submits to the backend, passing along the client-submitted request data, and transforming the data, if necessary. The HTTP method (or verb) and URI of the integration request are dictated by the backend (that is, the integration endpoint). They can be the same as or different from the method request's HTTP method and URI, respectively.
Authorize using Amazon Cognito that will reference the authentication table of DynamoDB - As an alternative to using IAM roles and policies or Lambda authorizers (formerly known as custom authorizers), you can use an Amazon Cognito user pool to control who can access your API in Amazon API Gateway.
To use an Amazon Cognito user pool with your API, you must first create an authorizer of the COGNITO_USER_POOLS type and then configure an API method to use that authorizer. After the API is deployed, the client must first sign the user in to the user pool, obtain an identity or access token for the user, and then call the API method with one of the tokens, which are typically set to the request's Authorization header. The API call succeeds only if the required token is supplied and the supplied token is valid, otherwise, the client isn't authorized to make the call because the client did not have credentials that could be authorized.
References:
https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-use-lambda-authorizer.html
https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-integrate-with-cognito.html
https://docs.aws.amazon.com/apigateway/latest/developerguide/how-to-integration-settings.html
https://docs.aws.amazon.com/apigateway/latest/developerguide/models-mappings.html

Question 37

A development team has noticed that one of the EC2 instances has been wrongly configured with the 'DeleteOnTermination' attribute set to True for its root EBS volume.
As a developer associate, can you suggest a way to disable this flag while the instance is still running?

- Set the DeleteOnTermination attribute to False using the command line
- Update the attribute using AWS management console. Select the EC2 instance and then uncheck the Delete On Termination check box for the root EBS volume
- Set the DisableApiTermination attribute of the instance using the API
- The attribute cannot be updated when the instance is running. Stop the instance from Amazon EC2 console and then update the flag

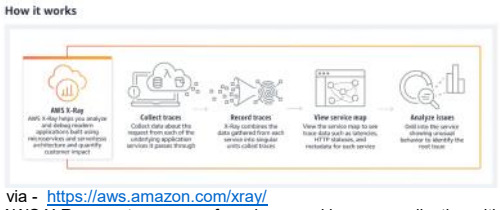
Overall explanation
Correct option:
When an instance terminates, the value of the DeleteOnTermination attribute for each attached EBS volume determines whether to preserve or delete the volume. By default, the DeleteOnTermination attribute is set to True for the root volume and is set to False for all other volume types.
Set the DeleteOnTermination attribute to False using the command line - If the instance is already running, you can set DeleteOnTermination to False using the command line.
Incorrect options:
Update the attribute using AWS management console. Select the EC2 instance and then uncheck the Delete On Termination check box for the root EBS volume - You can set the DeleteOnTermination attribute to False when you launch a new instance. It is not possible to update this attribute of a running instance from the AWS console.
Set the DisableApiTermination attribute of the instance using the API - By default, you can terminate your instance using the Amazon EC2 console, command-line interface, or API. To prevent your instance from being accidentally terminated using Amazon EC2, you can enable termination protection for the instance. The DisableApiTermination attribute controls whether the instance can be terminated using the console, CLI, or API. This option cannot be used to control the delete status for the EBS volume when the instance terminates.
The attribute cannot be updated when the instance is running. Stop the instance from Amazon EC2 console and then update the flag - This statement is wrong and given only as a distractor.

Question 38

A company uses microservices-based infrastructure to process the API calls from clients, perform request filtering and cache requests using the AWS API Gateway. Users report receiving 501 error code and you have been contacted to find out what is failing.
Which service will you choose to help you troubleshoot?

- Use API Gateway service
- Use CloudTrail service
- Use X-Ray service
- Use CloudWatch service

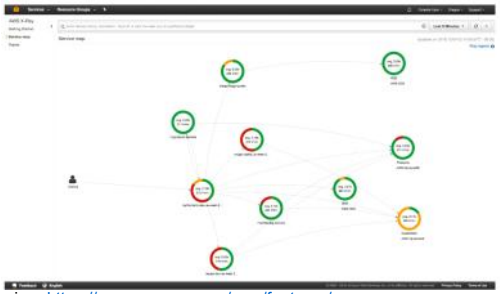
Overall explanation
Correct option:
Use X-Ray service - AWS X-Ray helps developers analyze and debug production, distributed applications, such as those built using a microservices architecture. With X-Ray, you can understand how your application and its underlying services are performing to identify and troubleshoot the root cause of performance issues and errors. X-Ray provides an end-to-end view of requests as they travel through your application, and shows a map of your application's underlying components. You can use X-Ray to analyze both applications in development and in production, from simple three-tier applications to complex microservices applications consisting of thousands of services.
X-Ray Overview:



via - https://aws.amazon.com/xray/

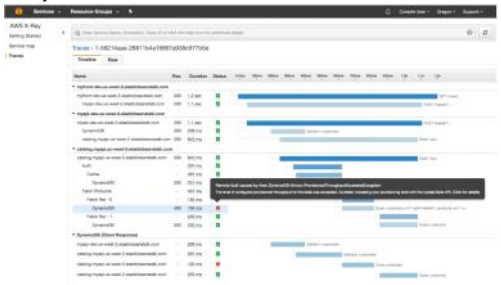
AWS X-Ray creates a map of services used by your application with trace data that you can use to drill into specific services or issues. This provides a view of connections between services in your application and aggregated data for each service, including average latency and failure rates. You can create dependency trees, perform cross-availability zone or region call detections, and more.

X-Ray Service maps:



via - https://aws.amazon.com/xray/features/

X-Ray Traces:



via - https://aws.amazon.com/xray/features/

Incorrect options:

Use CloudTrail service - With CloudTrail, you can get a history of AWS API calls for your account - including API calls made via the AWS Management Console, AWS SDKs, command-line tools, and higher-level AWS services (such as AWS CloudFormation). This is a very useful service for general monitoring and tracking. But, it will not give a detailed analysis of the outcome of microservices or drill into specific issues. For the current use case, X-Ray offers a better solution.

Use API Gateway service - Amazon API Gateway is an AWS service for creating, publishing, maintaining, monitoring, and securing REST, HTTP, and WebSocket APIs at any scale. API Gateway will not be able to drill into the flow between different microservices or their issues.

Use CloudWatch service - Amazon CloudWatch is a monitoring and management service that provides data and actionable insights for AWS, hybrid, and on-premises applications and infrastructure resources. CloudWatch can collect numbers and respond to AWS service-related events, but it can't help you debug microservices specific issues on AWS.